

PROFILING

◆ Idea

👉 Per capire dove un programma usa la maggior parte del tempo:

- bisogna fare **misurazioni**
- bisogna capire cosa limita le performance

👉 finora abbiamo usato funzioni di libreria per misurare i tempi

◆ Problema delle misurazioni

👉 domanda importante:

- quanto tempo ci mette `clock()` a eseguire?

👉 ogni chiamata a `clock()` :

- ha un costo
- non sempre richiede lo stesso tempo

✳ sotto una certa soglia le misure possono non avere senso

PERFORMANCE COUNTERS

◆ Idea

👉 I processori moderni hanno **performance counters** in hardware

👉 sono registri che contano eventi come:

- numero di istruzioni eseguite
- numero di cache miss
- altri eventi hardware

◆ Costo

👉 hanno costo prossimo a zero

👉 perché sono:

- contatori binari
- collegati a componenti già esistenti

◆ Utilizzo

👉 leggendo questi contatori a intervalli regolari possiamo capire:

- cosa sta facendo il programma
- dove perde performance

📌 PROFILING STATISTICO

◆ Idea

👉 Un profiler statistico prende dei **sample** del programma a intervalli regolari

👉 permette di capire:

- quanto tempo il programma passa in certe funzioni
- quali istruzioni sono più costose

◆ Tool usato

👉 negli esempi useremo:

✳ **perf**

👉 è fornito con Linux

👉 strumenti simili esistono anche su altri sistemi:

- ad esempio **VTune** di Intel

📌 PERF STAT

◆ Comando base

👉 Uso il comando base:

✳ **perf stat**

💻 Codice:

```
perf stat ./nome_programma
```

👉 fornisce statistiche come:

- numero di istruzioni eseguite
- istruzioni per ciclo di clock (**IPC**)
- numero e percentuale di **branch-misses**

◆ Primo test

👉 primo esperimento:

- confrontare una somma con branch
- confrontare una somma senza branch

✳ verrà fatto con codice C e analizzato con perf

PERF STAT PER LE CACHE

◆ Misurare cache references e cache misses



Codice:

```
perf stat -e cache-references,cache-misses ./nome_programma
```

👉 misura:

- accessi alla cache
- cache miss

◆ Limite

👉 questo comando dice quanti eventi sono avvenuti

👉 però NON dice:

✳ dove sono avvenuti nel codice

👉 per capire dove avvengono serve creare un report

PERF RECORD E PERF REPORT

◆ Registrare l'esecuzione

👉 Uso il comando:

✳ **perf record**

Codice:

```
perf record ./nome_programma
```

👉 crea un report dell'esecuzione

◆ Analizzare il report

👉 Uso il comando:

✳ **perf report**

Codice:

```
perf report
```

👉 apre un'interfaccia testuale

👉 permette di vedere:

- quali funzioni impiegano più tempo
- quali istruzioni pesano di più

SIMULAZIONE

◆ Idea

👉 approccio diverso dal prendere sample durante l'esecuzione

👉 invece di osservare il programma mentre gira:

✳️ si simula l'esecuzione

👉 si simulano:

- istruzioni che eseguono branch
- accessi alla memoria
- effetti sulle cache

◆ Vantaggi e svantaggi

👉 svantaggio:

- molto più lento

👉 vantaggio:

- possiamo simulare anche hardware diverso da quello della macchina reale

VALGRIND / CACHEGRIND

◆ Idea

👉 **Cachegrind** fa parte di Valgrind

👉 permette di simulare:

- uso della cache
- branch predictor

◆ Esempio pratico (flusso di utilizzo)

1. Esegue il programma con cachegrind



Codice:

```
valgrind --tool=cachegrind --branch-sim=yes ./nome_programma
```



Cosa succede:

- il programma viene eseguito
- vengono simulati cache e branch
- viene generato un file di output

2. Viene creato un file



Codice:

```
cachegrind.out.[numero]
```



Contiene:

- tutte le statistiche raccolte

3. Analizza il file



Codice:

```
cg_annotate cachegrind.out.[numero]
```

👉 mostra:

- statistiche per funzione

4. Analisi più dettagliata



Codice:

```
cg_annotate --auto=yes cachegrind.out.[numero]
```

👉 Mostra:

- statistiche riga per riga

✳ Utile per capire dove il codice è lento

5. Filtrare le statistiche



Codice:

```
cg_annotate --show=Dr,D1mr,D1mr,Bc,Bcm cachegrind.out.[numero]
```

👉 mostra solo alcune metriche:

- cache miss
- branch
- accessi memoria

◆ Cache simulate

👉 Cachegrind simula:

- **L1** (dati + istruzioni separati)
- **LL** (ultimo livello: L2, L3, ecc.)

PROFILING IN PYTHON

◆ cProfile

👉 Python ha un profiler integrato:

✳ cProfile

👉 è un profiler **deterministico**

◆ Limiti

👉 ha alcune limitazioni:

- difficile profilare codice nativo
- difficile profilare codice parallelo

◆ Scalene

👉 esistono profiler più moderni, ad esempio:

✳ Scalene

👉 permette profiling di:

- CPU
- memoria
- GPU
- codice nativo

Ora vediamo degli esempi pratici usati per fare profiling

ESEMPIO: BRANCH VS BRANCHLESS (SUM)

Idea

 Confrontiamo due versioni dello stesso algoritmo:

- con **branch (if)**
- senza branch (**branchless**)

 obiettivo:

 capire l'impatto dei branch sulle prestazioni

 questo si misura con:

- **perf stat**
- branch misses
- istruzioni eseguite

◆ Versione con branch



Codice:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n = 100000000;
    int *a = malloc(sizeof(int) * n);

    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 2;
    }

    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        if (a[i])
        {
            sum += a[i];
        }
    }

    printf("%d\n", sum);
    free(a);
    return 0;
}
```

◆ Spiegazione

👉 `a[i] = rand() % 2;`

- Genera valori 0 o 1
- Dati difficili da *predire*

👉 `if (a[i]) :`

- Introduce un **branch**
- La CPU deve fare predizione

★ Se la predizione è sbagliata:

- Pipeline svuotata
- Perdita di performance

◆ Versione branchless



Codice:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n = 100000000;
    int *a = malloc(sizeof(int) * n);

    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 2;
    }

    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += a[i];
    }

    printf("%d\n", sum);
    free(a);
    return 0;
}
```

◆ Spiegazione

👉 Non c'è `if`

👉 Ogni iterazione:

- Esegue sempre le stesse istruzioni

✳ niente branch → niente predizione

◆ Confronto con `perf`

👉 Usando il comando:

```
perf stat ./programma
```

possiamo osservare:

- **branch-misses**
- numero di istruzioni
- IPC

👉 Risultato tipico:

- Versione con *branch* → Si hanno più branch-misses
- Versione *branchless* → Meno errori

✳ Meno branch = Codice più stabile e veloce

◆ Osservazioni

👉 I branch sono costosi quando i dati sono *imprevedibili*

👉 Se i dati sono *prevedibili*:

- Il branch predictor funziona bene
- Il costo diminuisce

✳️ Quindi il problema dipende dai dati

◆ Conclusione

👉 Eliminare i branch:

- Può migliorare le prestazioni
- Soprattutto in loop grandi

Ora vediamo un caso più complesso: **binary search**

📌 ESEMPIO: BINARY SEARCH

◆ Idea

👉 Applichiamo lo stesso concetto a un algoritmo più complesso:

✳️ **binary search**

👉 vogliamo confrontare:

- versione con branch
- versione branchless

👉 Obiettivo:

✳️ capire l'impatto dei branch in un algoritmo reale

◆ Versione con branch



Codice:

```
int binary_search(int *a, int n, int key)
{
    int left = 0;
    int right = n - 1;

    while (left <= right)
    {
        int mid = (left + right) / 2;

        if (a[mid] == key)
        {
            return mid;
        }

        else if (a[mid] < key)
        {
            left = mid + 1;
        }

        else
        {
            right = mid - 1;
        }
    }

    return -1;
}
```

◆ Spiegazione

👉 Ad ogni iterazione:

- confronto `a[mid]` con `key`
- decisione tramite **if / else**

✳ ogni iterazione introduce branch

👉 Problema:

- I branch dipendono dai dati
- quindi possono essere imprevedibili

◆ Versione branchless



Codice:

```
int binary_search_branchless(int *a, int n, int key)
{
    int base = 0;

    for (int i = n / 2; i > 0; i /= 2)
    {
        int mid = base + i;
        int cond = (a[mid] <= key);
        base = cond ? mid : base;
    }

    if (a[base] == key)
    {
        return base;
    }

    return -1;
}
```

◆ Spiegazione

👉 Non usa `if` dentro il ciclo

👉 `cond = (a[mid] <= key) :`

- restituisce 0 o 1

👉 `base = cond ? mid : base :`

- aggiorna senza branch esplicito

✳ Usa operazioni aritmetiche al posto dei branch

◆ Confronto

👉 con `perf stat` possiamo osservare:

- branch misses
- numero di istruzioni
- IPC

👉 comportamento tipico:

- versione con branch → più branch-misses
- versione branchless → meno errori

✳️ meno branch = esecuzione più regolare

◆ Osservazione

👉 Rispetto al caso precedente:

- il codice è più complesso
- il beneficio dipende molto dai dati

👉 La versione branchless:

- può essere più veloce
- ma anche meno leggibile

◆ Conclusione

👉 Eliminare branch:

- può migliorare le prestazioni

- ma aumenta la complessità del codice

✳️ Quindi trade-off tra prestazioni e leggibilità